



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 11: Local Variables and Display Memory in Assembly Language

1. Objectives

- Understand the purpose and function of the stack frame.
- Learn to create, access, and destroy local variables within a subroutine.
- Understand the memory-mapped I/O model of the PC text-mode display.
- Learn to write data directly to video memory to display characters.
- Control character attributes (color, blinking) by manipulating the attribute byte.
- Write subroutines to perform common screen operations like `clrscr` and `printstr`.

2. Local Variables

Global variables exist for the *entire* program. While the Local variables are temporary "scratchpad" variables that exist *only* while a specific subroutine is running. Once the subroutine finishes, they are automatically destroyed, and their memory is reclaimed.

Key Concepts:

- Local variables are created by *decreasing the Stack Pointer (SP)* to make temporary space.
- They are accessed using *negative offsets from Base Pointer (BP)*.
- After the subroutine ends, the space is freed.

Why use them?

1. **Save Space:** The memory is reused by other subroutines.
2. **Avoid Conflicts:** If two subroutines use a global variable named `temp`, they will interfere with each other. If they each use a *local* variable named `temp`, there is no conflict.

How do we create them? We create local variables by making a "gap" on the stack just after saving `BP`. This "gap" and the parameters form the **stack frame**.



The Standard Subroutine Prologue (Setting up the frame):

```
my_subroutine:
    push bp          ; 1. Save the old base pointer
    mov  bp, sp      ; 2. Set our new frame's base pointer
    sub  sp, 4        ; 3. Make space for local variables
                      ; (e.g., 4 bytes for 2 words)
    ...
    ; (push registers you will use, e.g., push ax, push bx)
```

Accessing Data in the Stack Frame: The *BP* register is our anchor.

- **Parameters** are at *positive* offsets (e.g., [bp+4], [bp+6]).
- **Local Variables** are at *negative* offsets (e.g., [bp-2], [bp-4])

The Standard Subroutine Epilogue (Cleaning up):

```
...
; (pop registers in reverse order, e.g., pop bx, pop ax)
mov  sp, bp      ; 1. Destroy local variables
                      ; (restore sp to where bp is)
pop  bp          ; 2. Restore the old base pointer
ret  N           ; 3. Return (and clean up N bytes of parameters)
```

Example 1: Creating and Accessing a Local Variable

```
[org 0x0100]
jmp start

stacking:
    push bp
    mov  bp, sp
    sub  sp, 2          ; create 2-byte local variable

    mov  word [bp-2], 1234h ; store a value in local variable
    mov  ax, [bp-2]        ; load it into AX register

    mov  sp, bp
    pop  bp
    ret

start:
    call stacking
    mov  ax, 0x4c00
    int  0x21
```

Explanation: A temporary variable is created inside the subroutine, used, and then discarded before returning.



Example 2: Using Local Variable for Intermediate Calculations

```
[org 0x0100]
jmp start

multdiv:
    push bp
    mov bp, sp
    sub sp, 2                ; create local variable (2 bytes)

    mov ax, [bp+6]           ; first parameter
    mov bx, [bp+4]           ; second parameter
    mul bx                   ; ax = ax * bx

    mov [bp-2], ax           ; store product in local variable

    mov ax, [bp-2]           ; load product back into AX
    mov bx, 3                ; divisor
    div bx                   ; AX = AX / 3 (remainder in DX)

    mov [bp-2], ax           ; store quotient as final result

    mov sp, bp               ; clean stack space
    pop bp
    ret 4                    ; remove parameters

start:
    mov ax, 6
    push ax                  ; parameter 1
    mov ax, 4
    push ax                  ; parameter 2
    call multdiv
    mov ax, 0x4c00
    int 0x21
```

Example 3: Find largest number.

```
[org 0x0100]
jmp start

data: dw 10, 55, 2, 99, 45, 78
count: dw 6

; findMax(address, count)
; Finds the largest value in an array.
; Returns max value in ax
```



```
findMax:
    push bp
    mov bp, sp
    sub sp, 2                ; Make space for 1 local variable
(currentMax)

    push bx                  ; Save registers we will modify
    push cx
    push si

    mov si, [bp+6]           ; Get parameter 1: array address
    mov cx, [bp+4]           ; Get parameter 2: element count
    mov ax, [si]             ; Assume first element is max
    mov [bp-2], ax           ; Store in our local variable currentMax

    add si, 2                ; Point to second element
    dec cx                   ; Decrement count since we loaded one
    jz findMax_done          ; If count was 1, we are done

findMax_loop:
    mov ax, [si]             ; Get current element
    cmp ax, [bp-2]           ; Compare current element with currentMax
    jle findMax_next         ; If not greater, skip
    mov [bp-2], ax           ; It's greater, update currentMax

findMax_next:
    add si, 2                ; Move to next element in array
    loop findMax_loop        ; Repeat for all elements

findMax_done:
    mov ax, [bp-2]           ; Load return value from local variable

    pop si                   ; Restore registers
    pop cx
    pop bx

    mov sp, bp               ; Deallocate local variable
    pop bp                   ; Restore old base pointer
    ret 4                    ; Clean up 2 parameters (4 bytes)

start:
    mov ax, data
    push ax
    push word [count]
```



```
        call findMax          ; After this, ax will contain 99
mov     ax, 0x4c00
int     0x21
```

Example 4: Bubble Sort with Local Swap Flag

```
[org 0x0100]
jmp start
data: dw 10, 5, 8, 3, 12, 7

bubblesort:
    push bp
    mov  bp, sp
    sub  sp, 2
    push ax
    push bx
    push cx
    push si

    mov  bx, [bp+6]    ; array start
    mov  cx, [bp+4]    ; number of elements
    dec  cx
    shl  cx, 1

outer:
    mov  si, 0
    mov  word [bp-2], 0

inner:
    mov  ax, [bx+si]
    cmp  ax, [bx+si+2]
    jbe  noswap
    xchg ax, [bx+si+2]
    mov  [bx+si], ax
    mov  word [bp-2], 1

noswap:
    add  si, 2
    cmp  si, cx
    jne  inner
    cmp  word [bp-2], 1
    je   outer

    pop  si
    pop  cx
```



```

    pop    bx
    pop    ax
    mov    sp, bp
    pop    bp
    ret    4

start:
    mov    ax, data
    push   ax
    mov    ax, 6
    push   ax
    call   bubblesort
    mov    ax, 0x4c00
    int    0x21

```

3. Direct Display Memory (Drawing on the Screen)

In PC text mode, the screen is just a block of memory. Writing to this memory *instantly* updates the screen. This is called **memory-mapped I/O**.

Where is it? The memory block (Video Memory) starts at physical address B8000h. We access it using a segment register, typically *ES*.

```

    mov    ax, 0xb800
    mov    es, ax    ;Now 'es' points to the start of video memory

```

How is it structured? The screen is *80 columns wide by 25 rows high (80x25 grid)*. Each character on the screen is represented by **2 bytes (one word)** in memory.

- **Byte 1 (Low Byte):** The ASCII code of the character (e.g., 'A' is 0x41, ' ' is 0x20).
- **Byte 2 (High Byte):** The Attribute Byte (color, blinking).

The *Attribute Byte* controls the "look" of the character.

Bit(s)	Description	Example
7	Blinking	0 (No Blink), 1 (Blink)
6-4	Background Color (RGB)	000 (Black)
3	Foreground Intensity - Brightness	0 (Low Intensity)
2-0	Foreground Color (RGB)	111 (White)

Common Attributes:

- 0x07: Low-intensity White on Black (normal DOS text)
- 0x0F: High-intensity (Bright) White on Black
- 0x04: Red on Black
- 0x0A: Bright Green on Black
- 0x1E: Bright Yellow on Blue
- 0x87: Blinking White on Black



How to write a specific location (Row, Col)? The memory is linear. The offset from the start (es:0) is calculated: Offset (in bytes) = (Row * 80 + Col) * 2

- **Top-Left (Row 0, Col 0):** $(0 * 80 + 0) * 2 = 0$.
`mov word [es:0], 0x0741 ; Puts a White-on-Black 'A' at (0,0)`
- **Start of 2nd Row (Row 1, Col 0):** $(1 * 80 + 0) * 2 = 160$.
`mov word [es:160], 0x0442 ; Puts a Red-on-Black 'B' at (1,0)`

Example: `mov word [es:0], 0x0741`
→ Displays 'A' (41h) in white on black (07h) at top-left corner.

Important: For all tasks in this part, you *must* first point ES to the video memory:

```
mov ax, 0xb800
mov es, ax
```

Note: These programs *will not* show output inside the debugger. You must run the compiled .com file directly from the DOS prompt.

Example 5: Write a Single Character to the Screen

```
[org 0x0100]
mov es, 0xb800 ; video memory

mov word [es:0], 0x0741
; 'A' in white-on-black at row 0, col 0
mov ax, 0x4c00
int 0x21
```

Example 6: Display Characters in Different Colors

```
[org 0x0100]

mov ax, 0xb800
mov es, ax

mov word [es:0], 0x0F48 ; 'H' Bright white
mov word [es:2], 0x0F45 ; 'E'
mov word [es:4], 0x0F4C ; 'L'
mov word [es:6], 0x0F4C ; 'L'
mov word [es:8], 0x0F4F ; 'O'

mov ax, 0x4c00
int 0x21
```

Example 7: Writing to 2nd Row (Row 1, Col 0)

Offset = (Row * 80 + Col) * 2

For Row = 1, Col = 0: Offset = (1 * 80 + 0) * 2 = 160



```
[org 0x0100]

mov ax, 0xb800
mov es, ax

mov word [es:160], 0x0442
                                ; 'B' in red on black at row 1, col 0
mov ax, 0x4c00
int 0x21
```

Example 8: General Formula for Any Row, Any Column

This example writes 'X' in bright yellow on blue (1Eh) to Row 5, Column 10.

```
[org 0x0100]

mov ax, 0xb800
mov es, ax

mov bx, 5          ; Row
mov cx, 10         ; Col

; offset = (row * 80 + col) * 2
mov dx, bx
mov ax, 80
mul dx             ; ax = row * 80
add ax, cx         ; ax = row*80 + col
shl ax, 1          ; multiply by 2 (each char = 2 bytes)
;;;
mov word [es:ax], 0x1E58 ; 'X' (58h) yellow-on-blue (1Eh)

mov ax, 0x4c00
int 0x21
```

Example 9: Clear Entire Screen (fills with spaces)

```
[org 0x0100]

mov ax, 0xb800
mov es, ax
mov di, 0

next:
    mov word [es:di], 0x0720 ; space with white-on-black
    add di, 2
    cmp di, 4000             ; 80*25*2
    jne next
```



```
mov ax, 0x4c00  
int 0x21
```

Example 10: Print a Full String on Screen (Top-Left)

```
[org 0x0100]  
jmp start  
  
msg: db 'Hello Assembly!', 0  
len: dw 15  
  
print:  
    push bp  
    mov bp, sp  
    push es  
    push ax  
    push si  
    push di  
    push cx  
  
    mov ax, 0xb800  
    mov es, ax  
  
    mov si, [bp+6]    ; string address  
    mov cx, [bp+4]    ; length  
    mov di, 0         ; top-left position  
    mov ah, 0x0F      ; bright white  
  
next_char:  
    mov al, [si]       ; next ASCII character  
    mov [es:di], ax    ; write to video memory  
    add di, 2  
    inc si  
    loop next_char  
  
    pop cx  
    pop di  
    pop si  
    pop ax  
    pop es  
    pop bp  
    ret 4  
  
start:  
    mov ax, msg
```



```
    push ax
    push word [len]
    call print
mov ax, 0x4c00
int 0x21
```

Example 11: Clear screen (with clrscr using rep stows)

```
[org 0x0100]
jmp start
clrscr:
    push es
    push ax
    push di

    mov ax, 0xb800
    mov es, ax                ; point es to video base
    mov di, 0                 ; point di to top left (offset 0)
    mov ax, 0x0720            ; 0x07 = attribute, 0x20 = space
    mov cx, 2000              ; 80 * 25 = 2000 characters

    rep stosw                 ; "Repeat Store String Word"
                                ; This does:
                                ; 1. mov [es:di], ax
                                ; 2. add di, 2
                                ; 3. dec cx
                                ; 4. repeat if cx != 0

    pop di
    pop ax
    pop es
    ret
start:
    call clrscr                ; clear the screen first

    ; Now write something on the top-left corner
    mov ax, 0xb800
    mov es, ax

    mov word [es:0], 0x0F48 ; 'H' bright white
    mov word [es:2], 0x0F65 ; 'e'
    mov word [es:4], 0x0F6C ; 'l'
    mov word [es:6], 0x0F6C ; 'l'
    mov word [es:8], 0x0F6F ; 'o'
    mov ax, 0x4c00
    int 0x21
```



Example 12: Print a String Using Subroutine

```
[org 0x0100]
jmp start

message: db 'Assembly Rocks!'
length:  dw 15

printstr:
    push bp
    mov  bp, sp
    push es
    push ax
    push cx
    push si
    push di

    mov  ax, 0xb800
    mov  es, ax
    mov  si, [bp+6]
    mov  cx, [bp+4]
    mov  di, 0
    mov  ah, 0x0E

nextchar:
    mov  al, [si]
    mov  [es:di], ax
    add  di, 2
    add  si, 1
    loop nextchar

    pop  di
    pop  si
    pop  cx
    pop  ax
    pop  es
    pop  bp
    ret  4

start:
    mov  ax, message
    push ax
    push word [length]
    call printstr
```



```
mov ah, 0x0  
int 0x16
```

```
mov ax, 0x4c00  
int 0x21
```

4. Lab Tasks

Task 1: Write a subroutine *generateSquare* that:

- Receives a number N as a parameter.
- Uses *one local variable* to compute $N \times N$.
- Returns the result in AX.
- Must use a proper **stack frame** (push bp, mov bp, sp, etc.)

Task 2: Write a subroutine *sumArray(address, count)* that:

- Uses *at least one local variable* to store the running sum.
- Returns the total sum in AX
 - No global variables allowed.
 - Loop must use **SI** as the index and update it by **+2**.

Task 3: Write a program that:

- Takes *Row* in BX and *Column* in CX (hardcoded is allowed)
- Computes correct offset
- Displays the character '#' in **Bright White on Blue (1Eh)**

Task 4: Clear rows 5 to 7 only (3 rows, full width 80 columns) using:

- A loop
- Writing spaces (0x20)
- Attribute 07h
 - rep stosw is **NOT** allowed in this task.
 - Must compute appropriate start offsets.

Task 5: Write a subroutine: *printAt(stringAddress, length, row, col)*

That:

- Prints the string starting at the given row/column.
- Uses correct arithmetic to find the starting offset.
- Writes characters with attribute **0Fh** (bright white).

Task 6: Write a subroutine *reverseArray(address, count)* that:

- Reverses the array in-place.
- Uses one **local variable** for swapping elements.



- Updates both left & right pointers properly.

Test with an array of 6 numbers.

Task 7: Write a program that:

- Displays a horizontal line of **50 '=' characters**
- Starting at row 10, column 5
- With attribute **0x0A** (Bright Green on Black)

Task 8: Create a program that:

- Displays an '*' character
- Moves it left to right across row 12
- Updates its position every small delay
- Uses video memory only